

PARTE A – QUESTÕES OBJETIVAS

Questão 1 – b) Embedding

Armazenar o histórico dentro do próprio documento é a definição de embedding (dados aninhados).

Questão 2 – c) \$group

O \$group agrupa documentos por um campo (ex: origem) e permite contar registros com \$sum.

Questão 3 – c) Projeção

Projeção define quais campos retornar: find({}, { nome: 1, telefone: 1, _id: 0 }).

Questão 4 – c) skip() e limit()

O skip() pula registros já exibidos e o limit() define quantos retornar por página.

Questão 5 – b) \$exists

{ email: { \$exists: true } } retorna apenas documentos que possuem o campo email.

PARTE B – QUESTÕES DISCURSIVAS

Questão 1 – Embedding para histórico de contatos

A técnica utilizada é o Embedding, em que os dados relacionados são armazenados diretamente dentro do documento principal, formando um único objeto JSON. A vantagem é que a leitura é mais rápida e simples, pois todos os dados do lead e seu histórico são recuperados em uma única consulta, sem necessidade de joins ou lookups entre coleções. A limitação é que, se o histórico crescer indefinidamente, o documento pode ultrapassar o limite de 16 MB do MongoDB, além de tornar atualizações mais custosas, pois o documento inteiro precisa ser regravado.

Questão 2 – Referencing entre clientes e vendedores

O uso de Referencing (coleções separadas ligadas por `_id`) é mais adequado porque evita a duplicação de dados. Se todas as informações de um vendedor fossem repetidas em cada negociação, uma simples alteração de telefone ou nome exigiria atualizar centenas de documentos, criando inconsistências. Com referência, o vendedor existe em um único documento na coleção `vendedores`, e as negociações apenas armazenam seu `_id`. Isso garante integridade, facilita manutenção e reduz o tamanho total do banco de dados.

Questão 3 – Aggregation Framework para relatórios

O Aggregation Framework processa documentos em etapas (pipeline), transformando e agregando dados sem precisar exportá-los. Para obter a quantidade de leads por origem, utiliza-se `groupcomid:"group com _id: "`

`groupcomid:"origem"` e `$sum: 1`. Para leads por vendedor, agrupa-se por `vendedor_id` da mesma forma. Para negociações concluídas, aplica-se `$match` com `status: "concluida"` antes do `$group`. Outros operadores úteis são `$sort` para ordenar os resultados e `$project` para selecionar apenas os campos desejados no relatório.

Questão 4 – Índices no MongoDB

Índices são estruturas auxiliares (árvores B+) que o MongoDB mantém separadas da coleção, mapeando valores de campos para os documentos correspondentes. Sem índice, o banco realiza um `COLLSCAN`, lendo todos os documentos da coleção. Com índice, realiza um `IXSCAN`, navegando diretamente ao valor na árvore em tempo logarítmico. Os benefícios incluem redução drástica do tempo de consulta, menor uso de CPU e memória, e melhora na experiência do usuário final em sistemas com grande volume de dados.

Questão 5 – Duplicação de clientes e modelagem adequada

A duplicação ocorre quando não há validação de unicidade. A solução envolve duas frentes. A primeira é criar um índice único no campo identificador do cliente, como CPF ou e-mail, impedindo que o banco aceite dois documentos com o mesmo valor. A segunda é adotar Referencing: em vez de copiar dados do cliente dentro de leads e negociações, armazena-se apenas o `cliente_id`. Assim, o cliente existe em um único lugar, eliminando a raiz da duplicação estrutural e facilitando atualizações cadastrais.

Questão 6 – Tudo em uma única coleção

A vantagem dessa proposta é a simplicidade inicial de implementação e a facilidade de consulta quando os dados são sempre acessados juntos. As desvantagens são significativas: documentos com estruturas muito diferentes convivem na mesma coleção, dificultando validação, manutenção e leitura do código; consultas simples como listar apenas vendedores precisam filtrar por tipo, tornando-se mais lentas. Os impactos futuros incluem dificuldade para criar índices eficientes, prejudicando a escalabilidade horizontal (sharding). O padrão recomendado é separar entidades distintas em coleções próprias.

Questão 7 – Paginação com skip() e limit()

A paginação é importante porque retornar milhares de registros de uma só vez sobrecarrega o servidor, aumenta o tráfego de rede e degrada a experiência do usuário. O skip(n) descarta os n primeiros resultados, equivalente aos registros já exibidos nas páginas anteriores, enquanto o limit(m) restringe o retorno a m documentos, equivalente ao tamanho da página. Combinados com sort() para garantir uma ordem consistente, esses três métodos implementam a paginação completa: `.sort({ dataCadastro: -1 }).skip((pagina - 1) * 20).limit(20)`.

Questão 8 – Schema flexível (schemaless)

A ausência de esquema rígido permite adaptar a estrutura dos documentos conforme o sistema evolui, sem necessidade de migrações complexas. Diferentes leads podem ter campos distintos, como `perfil_social` para leads do Instagram e `pagina_visitada` para leads do site, refletindo fielmente a realidade de cada canal. Os cuidados necessários incluem realizar a validação na camada de aplicação ou via JSON Schema Validation do próprio MongoDB, pois sem essa disciplina o banco pode acumular dados inconsistentes, com campos com nomes diferentes representando a mesma informação, dificultando consultas e relatórios futuros.

Questão 9 – Embedding vs. Referencing

O Embedding armazena os dados relacionados dentro do próprio documento, oferecendo leitura rápida em uma única consulta, mas apresenta alta duplicação de dados e limitação de escalabilidade, pois o documento cresce indefinidamente. A manutenção é difícil, pois atualizar dados repetidos exige modificar vários documentos. O Referencing mantém os dados em coleções separadas, ligadas por `_id`, o que centraliza as informações, facilita a manutenção e oferece melhor escalabilidade. A desvantagem é que consultas podem exigir \$lookup, sendo um pouco mais lentas. Embedding é ideal para relações 1:1 ou 1:poucos; Referencing é preferível em relações 1:N com alto volume.

Questão 10 – Integração MongoDB + Node.js

Os documentos MongoDB são JSON, que é o formato nativo do JavaScript e do Node.js, eliminando camadas de conversão de dados. Isso acelera o desenvolvimento, reduz bugs de serialização e facilita a manutenção do código. Para os usuários finais, o ganho está na velocidade de resposta, pois o Node.js é assíncrono e não-bloqueante, processando múltiplas requisições simultâneas com eficiência. A combinação MongoDB com Node.js via Mongoose ou driver nativo é amplamente adotada em sistemas de CRM e plataformas de leads pela produtividade que oferece à equipe de desenvolvimento e pela escalabilidade que entrega em produção.

Questão 11 – Ranking de vendedores por negociações

A estratégia recomendada combina um índice composto nos campos status e vendedor_id para acelerar o filtro inicial, com um pipeline de agregação que usa \$match para selecionar apenas negociações concluídas, \$group para contar o total por vendedor, \$sort para ordenar do maior para o menor, \$limit para retornar apenas os mais relevantes e \$lookup para trazer o nome do vendedor da coleção correspondente. Essa abordagem garante que o relatório seja gerado com alto desempenho mesmo em grandes volumes de dados, sem necessidade de processar registros desnecessários.

Questão 12 – Escalabilidade com milhões de documentos

As principais decisões incluem adotar Referencing para entidades de alto crescimento como histórico e logs, evitando arrays ilimitados dentro de documentos. A indexação estratégica deve cobrir os campos mais consultados, como status, origem, vendedor_id e dataCadastro, com índices compostos para filtros frequentes com múltiplos campos. O sharding distribui a coleção entre múltiplos servidores usando uma chave adequada, evitando hotspots. TTL Indexes podem ser criados para expirar automaticamente dados temporários como logs. Evitar documentos acima de 1 MB também contribui diretamente para o desempenho de leitura e escrita.

Questão 13 – Embedding para histórico de interações

O uso de Embedding não é adequado como solução principal para o histórico completo de interações. O histórico tende a crescer indefinidamente, pois cada ligação, e-mail, visita ou mensagem gera um novo registro, podendo facilmente ultrapassar o limite de 16 MB do MongoDB. Além disso, toda escrita de um novo contato exige regravar o documento inteiro do cliente. A abordagem recomendada é o Referencing, criando uma coleção interacoes com cliente_id como referência. Opcionalmente, aplica-se o Subset Pattern, embutindo apenas as 3 ou 5 interações mais recentes no documento do cliente para leitura rápida, mantendo o histórico completo na coleção separada.

Questão 14 – MongoDB vs. Banco Relacional

O MongoDB armazena dados em documentos JSON com estrutura flexível, permitindo campos opcionais por documento e evolução do schema sem migrações complexas, enquanto bancos relacionais utilizam tabelas com colunas fixas que exigem ALTER TABLE a cada mudança. Em escalabilidade, o MongoDB suporta sharding horizontal nativo, enquanto bancos relacionais escalam principalmente de forma vertical. Para joins, bancos relacionais oferecem desempenho superior com JOINS nativos otimizados; o MongoDB utiliza \$lookup, que é menos eficiente. Em transações, bancos relacionais possuem suporte ACID mais maduro e testado. Para a 1000 Valle, cujos dados variam por canal e o sistema precisa evoluir rapidamente, o MongoDB é mais adequado.

Questão 15 – Parecer Técnico: Adoção do MongoDB

Recomendamos a adoção do MongoDB com base em três pilares técnicos. Primeiro, a flexibilidade de modelagem: leads captados por diferentes canais possuem estruturas de dados distintas, e o MongoDB permite que cada documento tenha campos específicos sem impacto nos demais, acelerando o desenvolvimento. Segundo, a escalabilidade horizontal: com o crescimento da base de clientes, o MongoDB suporta sharding nativo, distribuindo dados entre múltiplos servidores sem downtime, algo que bancos relacionais não oferecem nativamente. Terceiro, o desempenho em leitura: ao utilizar Embedding para dados consultados juntos, o sistema realiza uma única consulta ao banco, sem JOINS, resultando em respostas mais rápidas nos painéis comerciais. A integração nativa com Node.js e a sintaxe baseada em JSON também reduzem a curva de aprendizado da equipe.